深圳大学考试答题纸

（以论文、报告等形式考核专用）

二〇 二五 ～二〇 二六 学年度第 2 学期

课程编号 1503020001 课序号 01 课程名称 系统编程 主讲教师 冯禹洪 评分

学 号 2023150001 姓名 李文俊 专业年级 计算机科学与技术（创新班）2023

选题(一) 功能迭代

即时聊天系统

题号	1	2	3	4	5	6	7	8	9	10	基本题总分	附加题
得分												
评卷人												

教师评分补充说明：（请不要删除）

1. 答题撰写质量（10 分）

1.1. 代码运行截图基于华为 Taishan 服务器和 openEuler；

1.2. AIGC 代码片段必须附上 prompts；

1.3. 截图清晰，重点突出；

1.4. 图文并茂。

2. 服务器初始化（30 分）

统一说明：**本题各小题中要求截图代码的，都需要结合截图说明相关功能的实现**。

2.1.（5 分）请截图显示配置文件内容、服务器读取配置文件初始化系统的代码。

```
# === 服务器身份 ===
# 二进制名: chatserver_<short_name>_<version> = chatserver_lwj_1.4
server_name = chatserver
short_name  = lwj
version     = 1.4

# === 路径 ===
# 公共 FIFO 目录 FIFOFILES = ~/Server/fifo
fifo_dir        = /home/liwenjun2023150001/Server/fifo
# 客户端私有 FIFO 目录
client_fifo_dir = /home/liwenjun2023150001/Client/fifo
# 日志目录 LOGFILES = ~/log/chat-logs
log_dir         = /home/liwenjun2023150001/log/chat-logs

# === FIFO 命名前缀 ===
# 派生: lwj_reg_fifo / lwj_login_fifo / lwj_msg_fifo / lwj_logout_fifo
fifo_prefix = lwj

# === 线程池 POOLSIZE ===
# 启动时创建 100 个工作线程
poolsize    = 100
```

图 2.1a 配置文件 chatserver.conf 内容

配置文件 config/chatserver.conf：采用 key = value 文本格式，给出服务器名、缩写、版本、公共 FIFO 目录、客户端 FIFO 目录、日志目录、FIFO 前缀与线程池大小 POOLSIZE=100。

服务器调用点：服务器进程启动后的第一步就是读取配置——main() 调用 chat_config_load(argv[1], &g_cfg)，配置文件路径由命令行参数 argv[1] 传入而非写死。

```c
int chat_config_load(const char *path, ChatConfig *out)
{
    FILE *fp;
    char  line[1024];

    memset(out, 0, sizeof(*out));   /* 未出现的字段一律清零,避免读到栈上脏数据 */
    out->poolsize = 100;            /* 漏写 poolsize 也有合理默认值 */

    fp = fopen(path, "r");
    if (!fp) { perror("config: cannot open"); return -1; }   /* fail-fast */

    while (fgets(line, sizeof(line), fp)) {
        char *p = trim(line), *eq, *key, *val;
        if (*p == '\0' || *p == '#') continue;   /* 空行与整行注释跳过 */
        eq = strchr(p, '=');
        if (!eq) { fclose(fp); return -1; }      /* 缺 '=' 视为格式错误 */
        *eq = '\0';
        key = trim(p);  val = trim(eq + 1);      /* 以 '=' 切出 key / value */
        if (strcmp(key, "server_name") == 0)
            copy_field(out->server_name, sizeof(out->server_name), val, key);
        /* ... short_name / version / fifo_dir / log_dir / fifo_prefix 同理 ... */
        else if (strcmp(key, "poolsize") == 0)
            out->poolsize = (int)strtol(val, &endp, 10);
    }
    fclose(fp);
```

图 2.1b　chat_config_load() 解析配置的核心循环代码

配置文件解析：chat_config_load()是一个朴素而健壮的 key = value 解析器，进入时先 memset 清零整个 ChatConfig，再把 poolsize 预置为 100；主循环逐行 trim 去空白，跳过空行与 # 注释，用 strchr 定位 = 切出键值，按键分发写入对应字段。

可配置与 fail-fast：配置路径由 argv[1] 传入而非写死，同一份二进制可用不同 .conf 启动多个互不干扰的实例；fopen 失败或缺少 = 立即返回 -1，main 据此直接退出（fail-fast），宁可启动失败，也不让服务器带着残缺配置继续初始化 FIFO 与日志。

```c
    /* 二进制名(2.1.1):full_name = server_name_short_name_version */
    snprintf(out->full_name, sizeof(out->full_name), "%s_%s_%s",
             out->server_name, out->short_name, out->version);

    /* 公共命名管道(2.1.2):<fifo_dir>/<prefix>_{reg,login,msg,logout}_fifo */
    snprintf(fname, sizeof(fname), "%s_reg_fifo", out->fifo_prefix);
    join_path(out->fifo_register, n, "%s/%s", out->fifo_dir, fname);
    /* login / msg / logout 三个 FIFO 路径同理派生 */

    /* 日志文件(2.1.3):<log_dir>/server/{server,threads}.log */
    join_path(out->server_log_path,  n, "%s/%s", out->log_dir_server, "server.log");
    join_path(out->threads_log_path, n, "%s/%s", out->log_dir_server, "threads.log");
    return 0;
}
```

图 2.1c　由原始配置派生运行参数：二进制名、4 个公共 FIFO 全路径、两个日志文件路径

运行参数派生：把原始配置派生成服务器实际使用的运行参数，直接对应试题 2.1 的三个子项。改 fifo_prefix 或目录只需动配置文件，代码一行不用改；所有 snprintf/join_path 都做截断检查，路径过长即报错退出。

2.2.（5 分）请通过 Shell 命令 ps 获取服务器进程状态并附上运行结果截图，**结合进程状态信息和源代码实现说明服务器进程是守护进程**。通过 Shell 命令 ls 获取 4 个公共 FIFO 的信息并附上运行结果截图，**解释为何相应文件是 FIFO**。

2.2.1 服务器是守护进程的实现

守护进程化：服务器启动后立即守护进程化，在后台长期运行，不依赖任何终端。

```c
static int daemonize_step1_fork(void)
{
    pid_t pid = fork();
    if (pid < 0) {
        log_message("ERROR", "fork failed: %s", strerror(errno));
        return -1;
    }
    if (pid > 0) _exit(EXIT_SUCCESS);   /* 父进程退出,子进程被 init 收养 */
    if (setsid() == -1) {               /* 新建会话,脱离控制终端 */
        log_message("ERROR", "setsid failed: %s", strerror(errno));
        return -1;
    }
    return 0;
}
```

图 2.2a　daemonize_step1_fork():fork() 后父进程退出、子进程 setsid() 新建会话

```c
    if (daemonize_step1_fork() != 0) return EXIT_FAILURE;

    signal(SIGINT,  handle_signal);   /* 优雅退出:置标志,主循环再清理 */
    signal(SIGTERM, handle_signal);
    signal(SIGHUP,  SIG_IGN);         /* 忽略挂断信号 */
    signal(SIGPIPE, SIG_IGN);         /* 写已关闭 FIFO 不被杀死 */

    if (chdir("/") == -1) return EXIT_FAILURE;  /* 释放工作目录 */
    umask(0);                         /* 不让 umask 削减 mkfifo 权限 */
    open_logs(&g_cfg);
    redirect_std_to_devnull();        /* 标准流 → /dev/null */
```

图 2.2b　main() 中的守护化序列：信号处理、chdir("/")、umask(0)、重定向标准流

脱离终端的关键步骤：

① fork() 后**父进程退出**，子进程被 init 收养，从而脱离启动它的 shell

② setsid() 新建会话使其**没有控制终端**，不会收到终端信号

③ chdir("/") 释放工作目录，umask(0)保证随后 mkfifo 的权限不被掩码削减

④ 三个标准流重定向到 /dev/null

输出说明：ps 截图中该进程的 **PPID=1**（被 init 收养）、**TTY=?**（无控制终端），正是典型守护进程特征。

图 2.2c ps 命令显示服务器进程 PPID=1、TTY=?

2.2.2 4 个公共 FIFO 的创建与验证

```c
static int ensure_fifo(const char *path)
{
    struct stat st;
    if (stat(path, &st) == 0)            /* 已存在:必须是 FIFO,否则报错 */
        return S_ISFIFO(st.st_mode) ? 0 : -1;
    if (mkfifo(path, 0666) == -1) {      /* 不存在:创建命名管道,权限 0666 */
        log_message("ERROR", "mkfifo %s failed: %s", path, strerror(errno));
        return -1;
    }
    return 0;
}
```

图 2.2d ensure_fifo()：用 mkfifo() 创建命名管道，已存在则校验其确为 FIFO

公共命名管道创建：4 个公共管道（注册/登录/发消息/登出）用 `mkfifo()` 创建为**命名管道**，客户端按路径打开即可与服务器通信，无需共享父子关系。

输出说明：`ls -l` 结果中这 4 个文件的类型位为 **p**（`prw-rw-rw-`），`p` 即 *pipe*，说明它们是 FIFO 而非普通文件。

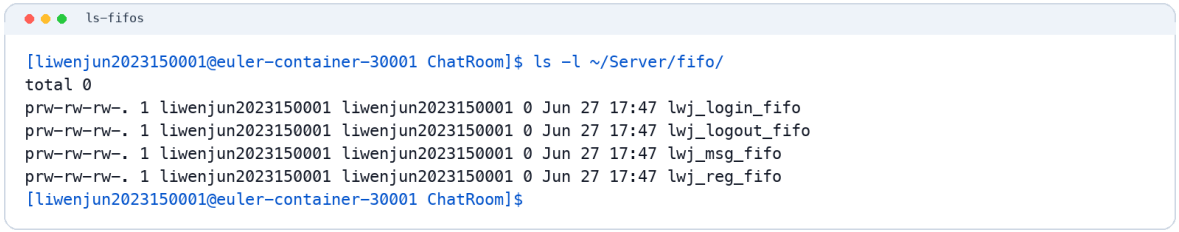

图 2.2e ls 命令显示 4 个公共 FIFO 文件

2.3.（5 分）请截图显示日志文件生成代码，运行以下 Shell 命令并截取运行结果：

```
$ls -l ~/log/chat-logs/server/server.log
$cat ~/log/chat-logs/server/server.log
```

日志文件生成：服务器把启动信息与运行事件写入 `server.log`、把线程池分派/回收写入 `threads.log`。两个文件均以 `O_CREAT|O_APPEND` 打开并强制 `0600` 权限。

```c
static int open_logs(const ChatConfig *cfg)
{
    mkdir_p(cfg->log_dir_server, 0700);                 /* 逐级创建日志目录树 */

    g_log_fd = open(cfg->server_log_path, O_CREAT | O_APPEND | O_WRONLY, 0600);
    if (g_log_fd == -1) return -1;
    fchmod(g_log_fd, 0600);                             /* 强制 0600,即便文件已存在 */

    g_threads_fd = open(cfg->threads_log_path, O_CREAT | O_APPEND | O_WRONLY, 0600);
    if (g_threads_fd == -1) return -1;
    fchmod(g_threads_fd, 0600);
    return 0;
}
```

图 2.3a　open_logs()：创建日志目录，以追加方式打开两个日志文件并 fchmod 0600

```c
static void vlog_to(int fd, const char *level, const char *fmt, va_list ap)
{
    char ts[32], line[1100];
    fmt_time((long)time(NULL), ts, sizeof(ts));         /* "YYYY-MM-DD HH:MM:SS" */
    int hdr = snprintf(line, sizeof(line), "%s [pid=%d] [%s] ",
                       ts, (int)getpid(), level);
    int body = vsnprintf(line + hdr, sizeof(line) - hdr, fmt, ap);
    size_t len = (size_t)hdr + (size_t)body;
    line[len++] = '\n';

    pthread_mutex_lock(&g_log_lock);                    /* 整行一次写出,避免交错 */
    write(fd, line, len);
    pthread_mutex_unlock(&g_log_lock);
}
```

图 2.3b　vlog_to()：加时间戳/pid，整行在互斥锁内一次 write()，保证多线程日志不交错

追加写与线程安全：

① O_APPEND 让每次写都原子地追加到文件末尾，重启不覆盖历史

② 工作线程并发写日志，故先把整行格式化到缓冲区，再在 g_log_lock 内**一次`write()`**——这样即使多个线程同时记日志也不会出现半行交错

③ 0600 限制只有属主可读写

输出说明：ls -l server.log 可见权限为 -rw------- (0600)； cat server.log 可见服务器启动时间、配置摘要、FIFO 路径、线程池构建（thread pool created: 100 threads）等信息。

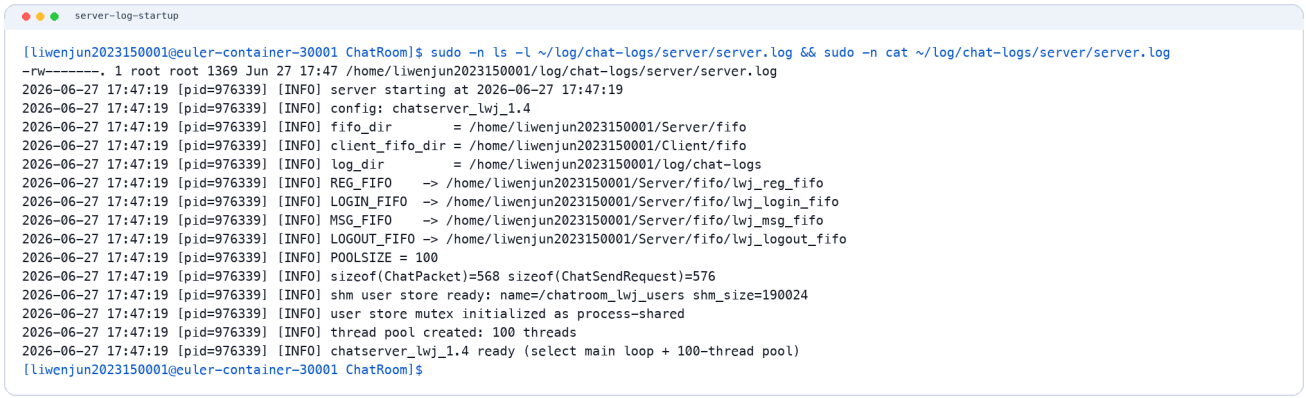

图 2.3c server.log 文件权限与启动日志

2.4.（10 分）请截图展示线程池线程管理代码：接收请求时分派线程，请求处理完后回收线程用于再分派，特别地，刚回收的线程倾向于下一次被分派。请给出线程池管理数据结构设计、线程池线程状态管理算法设计和实现等，并附上相应的运行展示。线程池中线程被分派和回收的时间记录在日志~/log/chat-logs/server/threads.log。

2.4.1 线程池数据结构设计

线程池构建：启动时创建恰好 POOLSIZE=100 个工作线程；主线程只负责 select+读请求+派发，业务逻辑在工作线程里跑。

```
1   typedef enum { WORKER_IDLE = 0, WORKER_BUSY = 1 } WorkerState;
2
3   typedef struct {            /* 单个工作线程 */
4       int             index;
5       pthread_t       tid;
6       WorkerState     state;
7       int             has_job;          /* 1 = 已分派待处理的 job */
8       void           *job;
9       pthread_mutex_t mu;               /* 保护 has_job / job */
10      pthread_cond_t  cv;               /* 唤醒该线程处理 job 或退出 */
11  } Worker;
12
13  struct ThreadPool {
14      int      size;
15      Worker  *workers;
16      int     *idle_stack;              /* 空闲 worker 下标,LIFO */
17      int      idle_top;                /* idle_stack[idle_top-1] 为栈顶 */
18      pthread_mutex_t stack_mu;         /* 保护 idle_stack / idle_top */
19  };
```

图 2.4a 线程池数据结构：每线程一把 mu/cv，空闲线程下标用整型数组 idle_stack 以栈(LIFO)管理

数据结构设计：空闲线程用**栈**，而不是队列管理——idle_top 指向栈顶，这样“**刚回收的线**

程在栈顶，下一次优先被分派”；每个线程独立的 mu/cv 让派发只唤醒目标线程，stack_mu 单独保护空闲栈，锁粒度小、互不干扰。

2.4.2 线程状态管理算法：LIFO 派发与回收

```c
static void idle_push(ThreadPool *p, int idx) {   /* 线程空闲 → 压栈顶 */
    pthread_mutex_lock(&p->stack_mu);
    p->idle_stack[p->idle_top++] = idx;
    pthread_mutex_unlock(&p->stack_mu);
}
static int idle_pop(ThreadPool *p) {              /* 取栈顶空闲线程;无则 -1 */
    int idx = -1;
    pthread_mutex_lock(&p->stack_mu);
    if (p->idle_top > 0) idx = p->idle_stack[--p->idle_top];
    pthread_mutex_unlock(&p->stack_mu);
    return idx;
}
```

图 2.4b　空闲栈的压入/弹出：idle_push 压栈顶，idle_pop 取栈顶（均加 stack_mu）

```c
int thread_pool_dispatch(ThreadPool *p, void *job)
{
    int idx = idle_pop(p);            /* LIFO:取最近回收的那个线程 */
    if (idx < 0) return -1;           /* 无空闲线程:立即返回"忙",不阻塞主循环 */
    Worker *w = &p->workers[idx];
    pthread_mutex_lock(&w->mu);
    w->state = WORKER_BUSY;  w->job = job;  w->has_job = 1;
    pthread_cond_signal(&w->cv);      /* 唤醒该工作线程去处理 */
    pthread_mutex_unlock(&w->mu);
    return 0;
}
```

图 2.4c　thread_pool_dispatch()：从栈顶取空闲线程并唤醒；无空闲立即返回 -1（非阻塞）

```c
        if (job) { p->handle_job(job, w->index, tid); p->free_job(job); }

        /* 处理完毕:标记空闲并压回 LIFO 栈,等待下一次派发。 */
        w->state = WORKER_IDLE;
        idle_push(p, w->index);
```

图 2.4d　worker_main() 处理完一个 job 后：置空闲并 idle_push 压回栈顶 → 实现“刚回收优先再派”

线程安全与性能：派发走 idle_pop、回收走 idle_push，二者都只在 stack_mu 临界区内动几个整数，开销极小；dispatch 在无空闲线程时**立即返回 -1**（主循环据此回“server busy”并丢弃），绝不阻塞 select 事件循环。LIFO 让最近活跃的线程被复用，其栈/缓存更可能仍是热的，利于性能。

2.4.3 分派与回收时间记录(threads.log)

```c
static void handle_job(void *job, int worker_index, unsigned long tid)
{
    ServerJob *j = (ServerJob *)job;
    threads_log("dispatch worker=#%d tid=%lu type=%s user=%s state=busy",
                worker_index, tid, job_type_name(j->kind), job_requester(j));
    switch (j->kind) {                /* 业务逻辑在工作线程里跑 */
    case JOB_REGISTER: do_register(&j->u.auth); break;
    case JOB_LOGIN:    do_login(&j->u.auth);    break;
    case JOB_LOGOUT:   do_logout(&j->u.logout); break;
    case JOB_CHAT:     do_chat(&j->u.chat);     break;
    }
    threads_log("recycle  worker=#%d tid=%lu type=%s state=idle",
                worker_index, tid, job_type_name(j->kind));
}
```

图 2.4e handle_job()：进入时记 dispatch...state=busy，业务跑完记 recycle...state=idle

输出说明：threads.log 中每处理一个请求会成对出现 dispatch ... state=busy 与 recycle ... state=idle，并带 worker=#编号、tid、type、时间戳，可直观看到线程被分派和回收的时刻。

```
threads-log
[liwenjun2023150001@euler-container-30001 ChatRoom]$ sudo -n tail -n 80 ~/log/chat-logs/server/threads.log
2026-06-27 17:47:24 [pid=976339] dispatch worker=#99 tid=281472968556960 type=register user=liwenjun_1 state=busy
2026-06-27 17:47:24 [pid=976339] recycle  worker=#99 tid=281472968556960 type=register state=idle
2026-06-27 17:47:27 [pid=976339] dispatch worker=#99 tid=281472968556960 type=register user=liwenjun_2 state=busy
2026-06-27 17:47:27 [pid=976339] recycle  worker=#99 tid=281472968556960 type=register state=idle
2026-06-27 17:47:30 [pid=976339] dispatch worker=#99 tid=281472968556960 type=register user=liwenjun_2 state=busy
2026-06-27 17:47:30 [pid=976339] recycle  worker=#99 tid=281472968556960 type=register state=idle
2026-06-27 17:47:33 [pid=976339] dispatch worker=#99 tid=281472968556960 type=register user=liwenjun_3 state=busy
2026-06-27 17:47:33 [pid=976339] recycle  worker=#99 tid=281472968556960 type=register state=idle
2026-06-27 17:47:38 [pid=976339] dispatch worker=#99 tid=281472968556960 type=login user=liwenjun_1 state=busy
2026-06-27 17:47:38 [pid=976339] recycle  worker=#99 tid=281472968556960 type=login state=idle
2026-06-27 17:47:39 [pid=976339] dispatch worker=#99 tid=281472968556960 type=login user=liwenjun_2 state=busy
2026-06-27 17:47:39 [pid=976339] recycle  worker=#99 tid=281472968556960 type=login state=idle
2026-06-27 17:47:40 [pid=976339] dispatch worker=#99 tid=281472968556960 type=login user=liwenjun_3 state=busy
2026-06-27 17:47:40 [pid=976339] recycle  worker=#99 tid=281472968556960 type=login state=idle
2026-06-27 17:47:42 [pid=976339] dispatch worker=#99 tid=281472968556960 type=chat user=liwenjun_1 state=busy
2026-06-27 17:47:42 [pid=976339] recycle  worker=#99 tid=281472968556960 type=chat state=idle
2026-06-27 17:47:42 [pid=976339] dispatch worker=#99 tid=281472968556960 type=chat user=liwenjun_1 state=busy
2026-06-27 17:47:42 [pid=976339] recycle  worker=#99 tid=281472968556960 type=chat state=idle
2026-06-27 17:47:42 [pid=976339] dispatch worker=#99 tid=281472968556960 type=chat user=liwenjun_1 state=busy
2026-06-27 17:47:42 [pid=976339] recycle  worker=#99 tid=281472968556960 type=chat state=idle
2026-06-27 17:47:43 [pid=976339] dispatch worker=#99 tid=281472968556960 type=chat user=liwenjun_1 state=busy
2026-06-27 17:47:43 [pid=976339] recycle  worker=#99 tid=281472968556960 type=chat state=idle
2026-06-27 17:47:43 [pid=976339] dispatch worker=#99 tid=281472968556960 type=chat user=liwenjun_1 state=busy
2026-06-27 17:47:43 [pid=976339] recycle  worker=#99 tid=281472968556960 type=chat state=idle
2026-06-27 17:47:45 [pid=976339] dispatch worker=#99 tid=281472968556960 type=chat user=liwenjun_1 state=busy
2026-06-27 17:47:45 [pid=976339] recycle  worker=#99 tid=281472968556960 type=chat state=idle
2026-06-27 17:47:46 [pid=976339] dispatch worker=#99 tid=281472968556960 type=logout user=liwenjun_2 state=busy
2026-06-27 17:47:46 [pid=976339] recycle  worker=#99 tid=281472968556960 type=logout state=idle
2026-06-27 17:47:47 [pid=976339] dispatch worker=#99 tid=281472968556960 type=chat user=liwenjun_1 state=busy
2026-06-27 17:47:47 [pid=976339] recycle  worker=#99 tid=281472968556960 type=chat state=idle
2026-06-27 17:47:48 [pid=976339] dispatch worker=#99 tid=281472968556960 type=login user=liwenjun_2 state=busy
2026-06-27 17:47:48 [pid=976339] recycle  worker=#99 tid=281472968556960 type=login state=idle
2026-06-27 17:47:51 [pid=976339] dispatch worker=#99 tid=281472968556960 type=chat user=liwenjun_1 state=busy
2026-06-27 17:47:51 [pid=976339] recycle  worker=#99 tid=281472968556960 type=chat state=idle
2026-06-27 17:47:52 [pid=976339] dispatch worker=#99 tid=281472968556960 type=chat user=liwenjun_1 state=busy
2026-06-27 17:47:52 [pid=976339] recycle  worker=#99 tid=281472968556960 type=chat state=idle
2026-06-27 17:47:53 [pid=976339] dispatch worker=#99 tid=281472968556960 type=chat user=liwenjun_1 state=busy
2026-06-27 17:47:53 [pid=976339] recycle  worker=#99 tid=281472968556960 type=chat state=idle
[liwenjun2023150001@euler-container-30001 ChatRoom]$
```

图 2.4f threads.log 中 dispatch/recycle 成对记录

2.5. （5 分）展示多路复用监听请求的代码、解释运行过程 select()、epoll()或 poll()的参数文件描述符集合的变化。

多路复用监听：主线程用 `select()` 同时监听 4 个公共 FIFO 的读端，哪个可读就读取并派发，单线程即可多路复用所有客户端请求。

```c
while (1) {
    FD_ZERO(&rfds);  maxfd = -1;
    for (i = 0; i < FIFO_COUNT; i++) {           /* 每轮都重建 fd_set */
        FD_SET(read_fds[i], &rfds);              /* 4 个公共 FIFO 读端入集合 */
        if (read_fds[i] > maxfd) maxfd = read_fds[i];
    }
    if (select(maxfd + 1, &rfds, NULL, NULL, NULL) == -1) {
        if (errno == EINTR) { shutdown_if_requested(); continue; }
        cleanup(EXIT_FAILURE);
    }
    /* 哪个 FIFO 就绪就读取并派发 */
    if (FD_ISSET(read_fds[FIFO_REGISTER], &rfds))
        drain_fifo(read_fds[FIFO_REGISTER], JOB_REGISTER);
    if (FD_ISSET(read_fds[FIFO_LOGIN], &rfds))
        drain_fifo(read_fds[FIFO_LOGIN], JOB_LOGIN);
    if (FD_ISSET(read_fds[FIFO_MESSAGE], &rfds))
        drain_fifo(read_fds[FIFO_MESSAGE], JOB_CHAT);
    if (FD_ISSET(read_fds[FIFO_LOGOUT], &rfds))
        drain_fifo(read_fds[FIFO_LOGOUT], JOB_LOGOUT);
}
```

图 2.5a　主事件循环：每轮重建 fd_set,select() 阻塞等待，返回后用 FD_ISSET 判断哪个 FIFO 可读

```c
static void drain_fifo(int fd, int kind) {
    for (;;) {                                   /* 把就绪 FIFO 一次性读空 */
        ServerJob job; job.kind = kind;
        ssize_t nread = read(fd, dst, reqsz);    /* 非阻塞读一条定长请求 */
        if (nread == -1) {
            if (errno == EAGAIN || errno == EWOULDBLOCK) return;  /* 读空,退出 */
            if (errno == EINTR) continue;
            return;
        }
        if (nread == 0) return;
        ServerJob *jp = malloc(sizeof(*jp)); *jp = job;
        if (thread_pool_dispatch(g_pool, jp) != 0) {  /* 派发给空闲工作线程 */
            reply_busy(&job); free(jp);          /* 无空闲线程:回 server busy */
        }
    }
}
```

图 2.5b　drain_fifo()：对就绪的 FIFO 非阻塞地读空所有请求，逐个打包派发给线程池

参数与文件描述符集合的变化：fd_set　rfds 是被监听描述符的位集合。**每一轮循环都先 `FD_ZERO` 再 `FD_SET`** 重新装入 4 个 FIFO 读端，这是因为 select() 返回时会**原地修改** rfds——把未就绪的位清掉，只留下就绪的位；若不重建，下一轮集合就不完整。select(maxfd+1, &rfds, NULL, NULL, NULL) 的第一个参数是最大 fd 加一，只监听读集合；返回后用 FD_ISSET 逐个测试 4 个读端是否就绪。

非阻塞读与保活写端：FIFO 读端以 O_RDONLY|O_NONBLOCK 打开，服务器另持一个**只写端**保持管道不被关闭（避免所有写者退出后 select 持续返回 EOF 就绪）；就绪后 drain_fifo 一次把该 FIFO 读空再回到 select，兼顾及时性与公平性。

3. 基本聊天功能展示（40 分）

统一说明：**本题各小题中要求截图代码的，都需要结合截图讨论数据结构设计、代码线程安全与运行性能**。

3.1 **（5 分）截图处理用户注册的代码**，并截屏显示以下运行过程：启动 3 个终端，分别用以下用户名依次注册：学生姓名拼音_1、学生姓名拼音_2、 学生姓名拼音_2。服务器将注册不成功的信息返回给相应的用户。

用户注册流程：客户端 /register 把（用户名、密码、自己的私有 FIFO 路径）打包写入公共注册 FIFO；服务器在工作线程中校验并登记，把结果回写到该客户端的私有 FIFO。

```c
/* 客户端:/register 把用户名/密码/自己的私有 FIFO 写入公共注册 FIFO */
static int send_auth_request(const char *server_fifo)
{
    ChatAuthRequest req;
    memset(&req, 0, sizeof(req));
    copy_string(req.username, sizeof(req.username), username);
    copy_string(req.password, sizeof(req.password), password);
    copy_string(req.fifo, sizeof(req.fifo), myfifo);   /* 回复要用的私有 FIFO */
    return write_request(server_fifo, &req, sizeof(req));
}
```

图 3.1a 客户端发起注册：把账号与私有 FIFO 写入公共注册 FIFO

```c
static void do_register(const ChatAuthRequest *req)   /* 工作线程里执行 */
{
    if (req->username[0] == '\0' || req->password[0] == '\0') {
        reply_to_fifo(req->fifo, 0, "username and password must not be empty");
        return;
    }
    UserStoreStatus st = user_store_register(&g_store, req->username,
                                             req->password, req->fifo, 0);
    switch (st) {
    case USER_STORE_OK:
        log_message("INFO", "(%s, register, %s)", req->username, tbuf);
        reply_to_fifo(req->fifo, 1, "register ok");                break;
    case USER_STORE_ERR_EXISTS:                                  /* 用户名重复 */
        reply_to_fifo(req->fifo, 0, "username already exists");    break;
    case USER_STORE_ERR_FULL:
        reply_to_fifo(req->fifo, 0, "server user table is full");  break;
    }
}
```

图 3.1b do_register()：校验非空，按 user_store_register 的返回分别回执成功/重复/表满

```c
UserStoreStatus user_store_register(UserStoreHandle *h, const char *username,
                                    const char *password, const char *fifo, int is_bot)
{
    su_lock(h);                             /* 共享内存互斥锁:整段查重+写入 */
    if (find_locked(s, username) != -1)
        ret = USER_STORE_ERR_EXISTS;        /* 已存在 → 拒绝,保证用户名唯一 */
    else if (s->user_count >= CHAT_MAX_USERS)
        ret = USER_STORE_ERR_FULL;
    else {
        ChatUserRecord *r = &s->users[s->user_count++];
        memset(r, 0, sizeof(*r));
        copy_string(r->username, sizeof(r->username), username);
        copy_string(r->password, sizeof(r->password), password);
        copy_string(r->fifo, sizeof(r->fifo), fifo);
        r->online = 0;  r->is_bot = is_bot ? 1 : 0;
        ret = USER_STORE_OK;
    }
    su_unlock(h);
    return ret;
}
```

图 3.1c　user_store_register()：在共享内存互斥锁内查重并写入用户记录

数据结构与线程安全：用户表是 POSIX 共享内存中的定长数组 `users[]`，查重 `find_locked` 与写入 `users[user_count++]` 必须在**同一把进程间互斥锁**内完成，否则两个并发注册可能都通过查重再写入同名记录；校验与回执文案分离，逻辑清晰。

输出说明：三个终端依次以 `liwenjun_1`、`liwenjun_2`、`liwenjun_2` 注册——前两个收到 `register ok`，第三个因用户名重复收到 `ERROR: username already exists`，正是服务器把注册失败信息回送给了对应用户。

register-liwenjun-1

```
[liwenjun2023150001@euler-container-30001 ChatRoom]$ ./bin/chatclient config/chatserver.conf liwenjun_1 pw1
client fifo: /home/liwenjun2023150001/Client/fifo/liwenjun_1
commands:
  /register             register current username and password
  /login                login current username and password
  /logout               logout current session (process keeps running)
  /send <user> <text>   send text to another online user (or bot)
  /bot add <x>          ask server to add x chat robots
  /bot del <x>          ask server to remove x online chat robots
  /quit                 exit client
chat> /register
chat>
[server] OK: register ok
chat>
```

图 3.1d-1 终端 1 注册 liwenjun_1 成功

```
register-liwenjun-2
[liwenjun2023150001@euler-container-30001 ChatRoom]$ ./bin/chatclient config/chatserver.conf liwenjun_2 pw2
client fifo: /home/liwenjun2023150001/Client/fifo/liwenjun_2
commands:
  /register             register current username and password
  /login                login current username and password
  /logout               logout current session (process keeps running)
  /send <user> <text>   send text to another online user (or bot)
  /bot add <x>          ask server to add x chat robots
  /bot del <x>          ask server to remove x online chat robots
  /quit                 exit client
chat> /register
chat>
[server] OK: register ok
chat>
```

图 3.1d-2 终端 2 注册 liwenjun_2 成功

```
register-duplicate-liwenjun-2
[liwenjun2023150001@euler-container-30001 ChatRoom]$ ./bin/chatclient config/chatserver.conf liwenjun_2 pw2
client fifo: /home/liwenjun2023150001/Client/fifo/liwenjun_2
commands:
  /register             register current username and password
  /login                login current username and password
  /logout               logout current session (process keeps running)
  /send <user> <text>   send text to another online user (or bot)
  /bot add <x>          ask server to add x chat robots
  /bot del <x>          ask server to remove x online chat robots
  /quit                 exit client
chat> /register
chat>
[server] ERROR: username already exists
chat>
```

图 3.1d-3 终端 3 重复注册 liwenjun_2 被拒绝

3.2 （5 分）截图处理用户**登录的代码**，截屏显示以下登录过程：启动 3 个终端，分别用以下用户名登录：学生姓名拼音_1、学生姓名拼音_2、 学生姓名拼音_3。服务器将成功登录的用户信息返回给登录成功的用户：在线的总人数，在线用户名；登录失败的信息返回给不成功的用户。

用户登录与在线名单：服务器校验账号密码；成功则把（在线总人数、在线用户名单）回执给登录者，再广播上线系统消息并刷新所有人的在线名单；失败则把原因回送给登录失败的用户。

```c
static void do_login(const ChatAuthRequest *req)
{
    UserStoreStatus st = user_store_login(&g_store, req->username,
                                          req->password, req->fifo, now);
    if (st == USER_STORE_OK) {
        char list[CHAT_TEXT_LEN], msg[CHAT_TEXT_LEN];
        int  count = 0;
        user_store_build_online_list(&g_store, req->username,
                                     list, sizeof(list), &count);
        snprintf(msg, sizeof(msg), "login ok; online %d: %s", count, list);
        ChatPacket p;
        make_packet(&p, CHAT_PACKET_REPLY, 1, "server", msg);
        p.online_count = count;  p.timestamp = now;    /* 回执带在线总数 */
        send_packet(req->fifo, &p);
        push_offline_messages(req->username, req->fifo);  /* 回推离线消息 */
        broadcast_online_list();            /* 广播刷新所有人的在线名单 */
        return;
    }
    if (st == USER_STORE_ERR_NOT_FOUND)
        reply_to_fifo(req->fifo, 0, "username does not exist");
    else if (st == USER_STORE_ERR_BAD_PASSWORD)
        reply_to_fifo(req->fifo, 0, "password is incorrect");
}
```

图 3.2a　do_login()：登录成功回执在线人数与名单，并触发离线消息回推与名单广播

```c
static void build_list_locked(const ChatUserStore *s, int vi,
                              char *buf, size_t bufsz, int *online_count)
{
    int count = 0;
    for (int i = 0; i < s->user_count; i++) {
        if (!s->users[i].online) continue;          /* 只统计在线用户 */
        count++;
        int important = (vi >= 0 && i != vi &&
                         s->send_count[vi][i] > CHAT_IMPORTANT_FRIEND_MIN);
        snprintf(buf + off, bufsz - off, "%s%s%s", off ? "," : "",
                 s->users[i].username, important ? "*" : "");  /* 重要朋友带 * */
    }
    if (online_count) *online_count = count;
}
```

图 3.2b　build_list_locked()：按观察者拼接在线名单，给“重要朋友”（累计发信 >5）加 *

个性化在线名单：在线名单按**观察者**个性化生成——send_count[vi][i] 记录观察者 vi 发给 i 的累计条数，超过 CHAT_IMPORTANT_FRIEND_MIN(5)即标 *，故同一份在线表对不同人显示不同的重要标记；整个统计在锁内完成，拿到的是一致快照。

输出说明：三个终端分别用 姓名_1、姓名_2、姓名_3 登录，各自收到形如 login ok; online 3: 姓名_1,姓名_2,姓名_3 的回执（在线总数 + 用户名）；若账号不存在或密码错误，则只有该用户收到 username does not exist / password is incorrect。

```
prep-register-liwenjun-3
[liwenjun2023150001@euler-container-30001 ChatRoom]$ ./bin/chatclient config/chatserver.conf liwenjun_3 pw3
client fifo: /home/liwenjun2023150001/Client/fifo/liwenjun_3
commands:
  /register             register current username and password
  /login                login current username and password
  /logout               logout current session (process keeps running)
  /send <user> <text>   send text to another online user (or bot)
  /bot add <x>          ask server to add x chat robots
  /bot del <x>          ask server to remove x online chat robots
  /quit                 exit client
chat> /register
chat>
[server] OK: register ok
chat>
```

图 3.2c-0 登录演示前置注册 liwenjun_3

```
login-liwenjun-1
[liwenjun2023150001@euler-container-30001 ChatRoom]$ ./bin/chatclient config/chatserver.conf liwenjun_1 pw1
client fifo: /home/liwenjun2023150001/Client/fifo/liwenjun_1
commands:
  /register             register current username and password
  /login                login current username and password
  /logout               logout current session (process keeps running)
  /send <user> <text>   send text to another online user (or bot)
  /bot add <x>          ask server to add x chat robots
  /bot del <x>          ask server to remove x online chat robots
  /quit                 exit client
chat> /login
chat>
[server] OK: login ok; online 1: liwenjun_1
chat>
[online 1] liwenjun_1
chat>
[system] liwenjun_2 is now online

[online 2] liwenjun_1,liwenjun_2
chat>
[system] liwenjun_3 is now online
chat>
[online 3] liwenjun_1,liwenjun_2,liwenjun_3
chat>
```

图 3.2c-1 终端 1 登录后看到在线人数与在线名单

```
login-liwenjun-2
[liwenjun2023150001@euler-container-30001 ChatRoom]$ ./bin/chatclient config/chatserver.conf liwenjun_2 pw2
client fifo: /home/liwenjun2023150001/Client/fifo/liwenjun_2
commands:
  /register             register current username and password
  /login                login current username and password
  /logout               logout current session (process keeps running)
  /send <user> <text>   send text to another online user (or bot)
  /bot add <x>          ask server to add x chat robots
  /bot del <x>          ask server to remove x online chat robots
  /quit                 exit client
chat> /login
chat>
[server] OK: login ok; online 2: liwenjun_1,liwenjun_2
chat>
[online 2] liwenjun_1,liwenjun_2
chat>
[system] liwenjun_3 is now online

[online 3] liwenjun_1,liwenjun_2,liwenjun_3
chat>
```

图 3.2c-2 终端 2 登录后看到在线人数与在线名单

```
login-liwenjun-3

[liwenjun2023150001@euler-container-30001 ChatRoom]$ ./bin/chatclient config/chatserver.conf liwenjun_3 pw3
client fifo: /home/liwenjun2023150001/Client/fifo/liwenjun_3
commands:
  /register             register current username and password
  /login                login current username and password
  /logout               logout current session (process keeps running)
  /send <user> <text>   send text to another online user (or bot)
  /bot add <x>          ask server to add x chat robots
  /bot del <x>          ask server to remove x online chat robots
  /quit                 exit client
chat> /login
chat>
[server] OK: login ok; online 3: liwenjun_1,liwenjun_2,liwenjun_3
chat>
[online 3] liwenjun_1,liwenjun_2,liwenjun_3
chat>
```

图 3.2c-3 终端 3 登录后看到在线人数与在线名单

3.3 （5 分）截图处理用户**消息发送**的代码：“学生姓名拼音_1”的用户给“学生姓名拼音_2”的用户发送信息“How are you，学生姓名拼音_2 ” 5 次。截取以上 5 次显示结果。

一对一消息发送：一对一发送，服务器把消息投递到接收方的私有 FIFO，并给发送方回执；同时累计“发送方→接收方”的消息条数。

```c
static void do_chat(const ChatSendRequest *req)
{
    if (strcmp(req->to, CHAT_BOTMGR_TARGET) == 0) { do_bot_manager(req); return; }

    ChatSendInfo info;
    user_store_prepare_send(&g_store, req->from, req->to, &info);  /* 锁内取齐双方状态 */
    if (!info.sender_online) return;              /* 发送方未登录:丢弃 */
    if (!info.target_exists) {
        reply_to_fifo(info.from_fifo, 0, "target user does not exist");
        return;
    }
    if (info.target_is_bot) { do_chat_to_bot(req->from, info.from_fifo,
                                             req->to, &info); return; }
    do_chat_to_human(req, &info);
}
```

图 3.3a do_chat()：先分流机器人管理目标，再取齐双方状态，按 在线/离线/机器人 走不同分支

```c
static void do_chat_to_human(const ChatSendRequest *req, const ChatSendInfo *info)
{
    if (info->target_online) {
        ChatPacket msg;
        make_packet(&msg, CHAT_PACKET_MESSAGE, 1, req->from, req->text);
        msg.timestamp = now;
        if (send_packet(info->to_fifo, &msg) == 0) {   /* 投递到对方私有 FIFO */
            int c = user_store_increment_send(&g_store, req->from, req->to);
            int important = (c > CHAT_IMPORTANT_FRIEND_MIN);  /* 累计 >5 即重要朋友 */
            char ack[CHAT_TEXT_LEN];
            snprintf(ack, sizeof(ack), "message sent to %s%s",
                     req->to, important ? "*" : "");
            ChatPacket rp;
            make_packet(&rp, CHAT_PACKET_REPLY, 1, "server", ack);
            rp.send_count = c;  rp.timestamp = now;    /* 回执带累计发送次数 */
            send_packet(info->from_fifo, &rp);
            return;
        }
        user_store_set_offline(&g_store, req->to, info->to_fifo); /* 投递失败→置离线 */
    }
    /* 目标离线:转入离线暂存(见 3.4) */
}
```

图 3.3b　do_chat_to_human()：在线则投递并回执；投递失败即判定对方掉线转离线

线程安全与性能：user_store_prepare_send 在**一次加锁**内取齐双方的在线状态、私有 FIFO、是否机器人，复制到本地 info 后**在锁外做 FIFO I/O**——临界区短、不在持锁时阻塞写管道；send_count[from][to] 是二维计数矩阵，user_store_increment_send 原子自增并返回累计值。

输出说明：姓名_1 给 姓名_2 连发 5 次 How are you，姓名_2，每次都收到 message sent to 姓名_2 回执，接收端逐条显示 [姓名_1] How are you，姓名_2；当累计**超过 5 次**（第 6 次起），回执与在线名单中该好友会带上 *（重要朋友）。

send-five-sender

```
[liwenjun2023150001@euler-container-30001 ChatRoom]$ ./bin/chatclient config/chatserver.conf liwenjun_1 pw1
client fifo: /home/liwenjun2023150001/Client/fifo/liwenjun_1
commands:
  /register             register current username and password
  /login                login current username and password
  /logout               logout current session (process keeps running)
  /send <user> <text>   send text to another online user (or bot)
  /bot add <x>          ask server to add x chat robots
  /bot del <x>          ask server to remove x online chat robots
  /quit                 exit client
chat> /login
chat>
[server] OK: login ok; online 1: liwenjun_1
chat>
[online 1] liwenjun_1
chat>
[system] liwenjun_2 is now online

[online 2] liwenjun_1,liwenjun_2
chat>
[system] liwenjun_3 is now online
chat>
[online 3] liwenjun_1,liwenjun_2,liwenjun_3
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat>
```

图 3.3c-1 发送方连续发送 5 条消息并收到 5 条回执

send-five-receiver

```
[liwenjun2023150001@euler-container-30001 ChatRoom]$ ./bin/chatclient config/chatserver.conf liwenjun_2 pw2
client fifo: /home/liwenjun2023150001/Client/fifo/liwenjun_2
commands:
  /register             register current username and password
  /login                login current username and password
  /logout               logout current session (process keeps running)
  /send <user> <text>   send text to another online user (or bot)
  /bot add <x>          ask server to add x chat robots
  /bot del <x>          ask server to remove x online chat robots
  /quit                 exit client
chat> /login
chat>
[server] OK: login ok; online 2: liwenjun_1,liwenjun_2
chat>
[online 2] liwenjun_1,liwenjun_2
chat>
[system] liwenjun_3 is now online

[online 3] liwenjun_1,liwenjun_2,liwenjun_3
chat>
[liwenjun_1] How are you, liwenjun_2
chat>
[liwenjun_1] How are you, liwenjun_2
chat>
[liwenjun_1] How are you, liwenjun_2
chat>
[liwenjun_1] How are you, liwenjun_2
chat>
[liwenjun_1] How are you, liwenjun_2
chat>
```

图 3.3c-2 接收方收到来自 liwenjun_1 的 5 条消息

```
important-friend-star

[liwenjun2023150001@euler-container-30001 ChatRoom]$ ./bin/chatclient config/chatserver.conf liwenjun_1 pw1
client fifo: /home/liwenjun2023150001/Client/fifo/liwenjun_1
commands:
  /register             register current username and password
  /login                login current username and password
  /logout               logout current session (process keeps running)
  /send <user> <text>   send text to another online user (or bot)
  /bot add <x>          ask server to add x chat robots
  /bot del <x>          ask server to remove x online chat robots
  /quit                 exit client
chat> /login
chat>
[server] OK: login ok; online 1: liwenjun_1
chat>
[online 1] liwenjun_1
chat>
[system] liwenjun_2 is now online

[online 2] liwenjun_1,liwenjun_2
chat>
[system] liwenjun_3 is now online
chat>
[online 3] liwenjun_1,liwenjun_2,liwenjun_3
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2*
chat>
```

图 3.3d 第 6 次成功发送后回执中 liwenjun_2 带 *，证明重要朋友标记

3.4 （10 分）截图处理用户**退出、离线用户信息保存，登录后消息抵达等代码：**名为“学生姓名拼音_2”的用户退出，服务器将其退出的信息及剩余在线用户的信息显示给在线用户。然后，“学生姓名拼音_1”的用户给“学生姓名拼音_2”发送信息“Hi，let’s play badminton? ”。最后名为“学生姓名拼音_2”的用户登录系统，收到前面的信息，展示发送的时间。

3.4.1 用户退出

```
static void do_logout(const ChatLogoutRequest *req)
{
    UserStoreStatus st = user_store_logout(&g_store, req->username, req->fifo, now);
    if (st != USER_STORE_OK) return;       /* 不存在 / FIFO 不匹配:静默丢弃 */

    reply_to_fifo(req->fifo, 1, "logout ok");
    log_message("INFO", "(%s, logout, %s)", req->username, tbuf);
    char sys[128];
    snprintf(sys, sizeof(sys), "%s has logged out", req->username);
    broadcast_system(sys, NULL);           /* 通知其余在线用户该用户退出 */
    broadcast_online_list();               /* 刷新在线名单 */
}
```

图 3.4a　do_logout()：校验通过后置离线，向其余在线用户广播退出并刷新名单

防伪登出与广播：登出请求要求 username 与登记的私有 fifo **都匹配**才生效（user_store_logout 内校验），可防止伪造他人登出；该用户已离线，广播时不在接收集合中，故其余在线用户都会看到“xxx has logged out”与新名单。

3.4.2 离线消息暂存

```
UserStoreStatus user_store_store_offline(UserStoreHandle *h, const char *from,
                                         const char *to, const char *text, long ts)
{
    su_lock(h);
    for (int i = 0; i < CHAT_MAX_OFFLINE_MESSAGES; i++) {
        if (s->offline_messages[i].used) continue;   /* 找一个空槽 */
        s->offline_messages[i].used = 1;
        copy_string(s->offline_messages[i].from, CHAT_NAME_LEN, from);
        copy_string(s->offline_messages[i].to,   CHAT_NAME_LEN, to);
        copy_string(s->offline_messages[i].text, CHAT_TEXT_LEN, text);
        s->offline_messages[i].timestamp = ts;       /* 记录原始发送时间 */
        ret = USER_STORE_OK;  break;
    }
    su_unlock(h);
    return ret;
}
```

图 3.4b　user_store_store_offline()：在共享内存定长队列里找空槽，连同原始时间戳暂存

离线消息暂存：当接收方不在线（或刚被判定掉线）时，do_chat_to_human 调用 user_store_store_offline 在共享内存的定长离线队列里找一个空槽，把 from/to/text 连同原始发送时间戳一起写入，等其重新登录后再由下一节的逻辑回推。

3.4.3 重新登录后离线消息抵达

```c
static void push_offline_messages(const char *user, const char *user_fifo)
{
    ChatOfflineMessage *msgs = malloc(sizeof(*msgs) * CHAT_MAX_OFFLINE_MESSAGES);
    /* 取出即清空:一次加锁内拿到发给 user 的全部离线消息 */
    int n = user_store_pop_offline(&g_store, user, msgs, CHAT_MAX_OFFLINE_MESSAGES);
    for (int i = 0; i < n; i++) {
        ChatPacket p;
        make_packet(&p, CHAT_PACKET_OFFLINE_PUSH, 1, msgs[i].from, msgs[i].text);
        p.timestamp = msgs[i].timestamp;       /* 保留原始发送时间,客户端据此显示 */
        send_packet(user_fifo, &p);
    }
    free(msgs);
}
```

图 3.4c　push_offline_messages()：登录时取出并清空该用户的离线消息，带原始时间逐条回推

离线队列与原子取出：离线消息存放在共享内存中的定长数组，每条带 from/to/text/timestamp；把原来的“peek+clear”合并为一次 user_store_pop_offline——在**同一把锁内取出并清槽**(copy-and-clear)，避免取出和清除之间的竞态；回推时用消息的**原始** timestamp，客户端就能显示消息真正的发送时间。

输出说明：姓名_2 退出后，在线用户看到其退出信息与剩余在线名单；此时 姓名_1 发 Hi, let's play badminton? 收到 target offline; message stored as pending; 待 姓名_2 重新登录，立即收到该离线消息，并显示其**原始发送时间。**

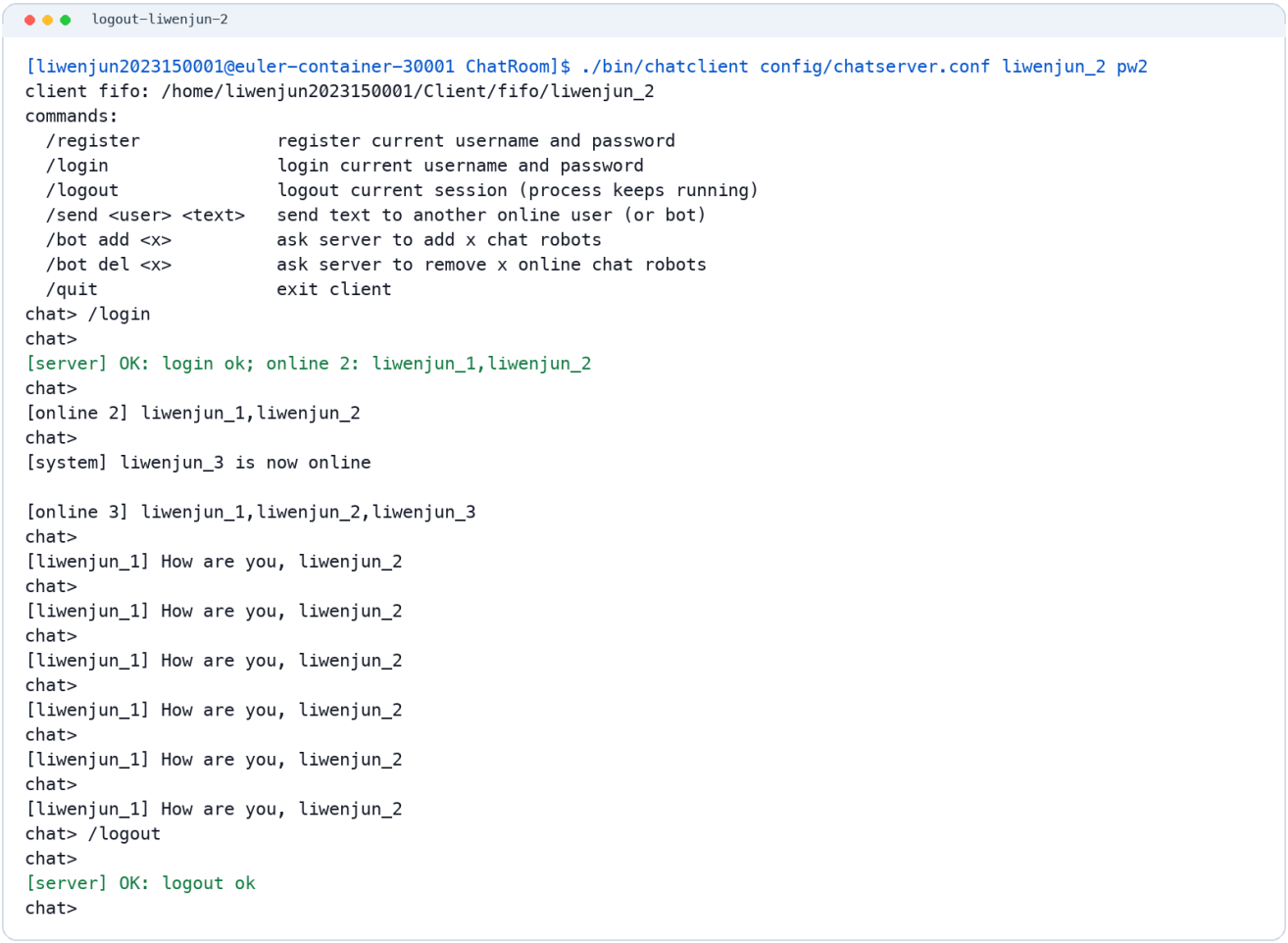

图 3.4d-1 终端 2 执行 logout 退出系统

offline-store-sender

```
[liwenjun2023150001@euler-container-30001 ChatRoom]$ ./bin/chatclient config/chatserver.conf liwenjun_1 pw1
client fifo: /home/liwenjun2023150001/Client/fifo/liwenjun_1
commands:
  /register             register current username and password
  /login                login current username and password
  /logout               logout current session (process keeps running)
  /send <user> <text>   send text to another online user (or bot)
  /bot add <x>          ask server to add x chat robots
  /bot del <x>          ask server to remove x online chat robots
  /quit                 exit client
chat> /login
chat>
[server] OK: login ok; online 1: liwenjun_1
chat>
[online 1] liwenjun_1
chat>
[system] liwenjun_2 is now online

[online 2] liwenjun_1,liwenjun_2
chat>
[system] liwenjun_3 is now online
chat>
[online 3] liwenjun_1,liwenjun_2,liwenjun_3
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2*
chat>
[system] liwenjun_2 has logged out

[online 2] liwenjun_1,liwenjun_3
chat> /send liwenjun_2 Hi, let's play badminton?
chat>
[server] ERROR: target offline; message stored as pending
chat>
```

图 3.4d-2 终端 1 看到 liwenjun_2 退出并发送离线消息暂存

offline-push-relogin

```
[liwenjun2023150001@euler-container-30001 ChatRoom]$ ./bin/chatclient config/chatserver.conf liwenjun_2 pw2
client fifo: /home/liwenjun2023150001/Client/fifo/liwenjun_2
commands:
  /register             register current username and password
  /login                login current username and password
  /logout               logout current session (process keeps running)
  /send <user> <text>   send text to another online user (or bot)
  /bot add <x>          ask server to add x chat robots
  /bot del <x>          ask server to remove x online chat robots
  /quit                 exit client
chat> /login
chat>
[server] OK: login ok; online 2: liwenjun_1,liwenjun_2
chat>
[online 2] liwenjun_1,liwenjun_2
chat>
[system] liwenjun_3 is now online

[online 3] liwenjun_1,liwenjun_2,liwenjun_3
chat>
[liwenjun_1] How are you, liwenjun_2
chat>
[liwenjun_1] How are you, liwenjun_2
chat>
[liwenjun_1] How are you, liwenjun_2
chat>
[liwenjun_1] How are you, liwenjun_2
chat>
[liwenjun_1] How are you, liwenjun_2
chat>
[liwenjun_1] How are you, liwenjun_2
chat>
[liwenjun_1] How are you, liwenjun_2
chat> /logout
chat>
[server] OK: logout ok
chat> /login
chat>
[server] OK: login ok; online 3: liwenjun_1,liwenjun_2,liwenjun_3
chat>
[offline from liwenjun_1 @ 2026-06-27 17:47:47] Hi, let's play badminton?

[online 3] liwenjun_1,liwenjun_2,liwenjun_3
chat>
```

图 3.4d-3 终端 2 重新登录后收到带发送时间的离线消息

3.5 （15 分）查看服务器日志信息：

$cat ~/log/chat-logs/server/server.log

温馨提醒：包括上述过程中所有客户端用户的注册、登录、发信息、退出、用户再次登录后收到离线消息、运行期线程管理等内容。

服务器运行日志：服务器把整个会话过程写入 ~/log/chat-logs/server/server.log。每条业务事件统一记成 时间 [pid] [级别] (用户, 动作, 时间戳) 的结构化文本，动作涵盖 register / login / sent / pending / logout / offline push 等。

输出说明：cat server.log 可完整复现上述过程——3 个用户的注册与登录、姓名_1 给 姓名_2

的 5 次发送、姓名_2 退出、离线消息 pending、姓名_2 再次登录后的 offline push，以及运行期线程池的 dispatch/recycle（记录在 threads.log）。结构化的（用户、动作、时间）三元组让批阅者能逐条核对每个功能点。

```
server-log-full

[liwenjun2023150001@euler-container-30001 ChatRoom]$ sudo -n cat ~/log/chat-logs/server/server.log
2026-06-27 17:47:19 [pid=976339] [INFO] server starting at 2026-06-27 17:47:19
2026-06-27 17:47:19 [pid=976339] [INFO] config: chatserver_lwj_1.4
2026-06-27 17:47:19 [pid=976339] [INFO] fifo_dir        = /home/liwenjun2023150001/Server/fifo
2026-06-27 17:47:19 [pid=976339] [INFO] client_fifo_dir = /home/liwenjun2023150001/Client/fifo
2026-06-27 17:47:19 [pid=976339] [INFO] log_dir         = /home/liwenjun2023150001/log/chat-logs
2026-06-27 17:47:19 [pid=976339] [INFO] REG_FIFO    -> /home/liwenjun2023150001/Server/fifo/lwj_reg_fifo
2026-06-27 17:47:19 [pid=976339] [INFO] LOGIN_FIFO  -> /home/liwenjun2023150001/Server/fifo/lwj_login_fifo
2026-06-27 17:47:19 [pid=976339] [INFO] MSG_FIFO    -> /home/liwenjun2023150001/Server/fifo/lwj_msg_fifo
2026-06-27 17:47:19 [pid=976339] [INFO] LOGOUT_FIFO -> /home/liwenjun2023150001/Server/fifo/lwj_logout_fifo
2026-06-27 17:47:19 [pid=976339] [INFO] POOLSIZE = 100
2026-06-27 17:47:19 [pid=976339] [INFO] sizeof(ChatPacket)=568 sizeof(ChatSendRequest)=576
2026-06-27 17:47:19 [pid=976339] [INFO] shm user store ready: name=/chatroom_lwj_users shm_size=190024
2026-06-27 17:47:19 [pid=976339] [INFO] user store mutex initialized as process-shared
2026-06-27 17:47:19 [pid=976339] [INFO] thread pool created: 100 threads
2026-06-27 17:47:19 [pid=976339] [INFO] chatserver_lwj_1.4 ready (select main loop + 100-thread pool)
2026-06-27 17:47:24 [pid=976339] [INFO] request arrival: type=register user=liwenjun_1
2026-06-27 17:47:24 [pid=976339] [INFO] (liwenjun_1, register, 2026-06-27 17:47:24)
2026-06-27 17:47:27 [pid=976339] [INFO] request arrival: type=register user=liwenjun_2
2026-06-27 17:47:27 [pid=976339] [INFO] (liwenjun_2, register, 2026-06-27 17:47:27)
2026-06-27 17:47:30 [pid=976339] [INFO] request arrival: type=register user=liwenjun_2
2026-06-27 17:47:30 [pid=976339] [WARN] duplicate register rejected: liwenjun_2
2026-06-27 17:47:33 [pid=976339] [INFO] request arrival: type=register user=liwenjun_3
2026-06-27 17:47:33 [pid=976339] [INFO] (liwenjun_3, register, 2026-06-27 17:47:33)
2026-06-27 17:47:38 [pid=976339] [INFO] request arrival: type=login user=liwenjun_1
2026-06-27 17:47:38 [pid=976339] [INFO] (liwenjun_1, login, 2026-06-27 17:47:38) online=1
2026-06-27 17:47:39 [pid=976339] [INFO] request arrival: type=login user=liwenjun_2
2026-06-27 17:47:39 [pid=976339] [INFO] (liwenjun_2, login, 2026-06-27 17:47:39) online=2
2026-06-27 17:47:40 [pid=976339] [INFO] request arrival: type=login user=liwenjun_3
2026-06-27 17:47:40 [pid=976339] [INFO] (liwenjun_3, login, 2026-06-27 17:47:40) online=3
2026-06-27 17:47:42 [pid=976339] [INFO] request arrival: type=chat user=liwenjun_1
2026-06-27 17:47:42 [pid=976339] [INFO] (liwenjun_1, liwenjun_2, 2026-06-27 17:47:42, sent)
2026-06-27 17:47:42 [pid=976339] [INFO] request arrival: type=chat user=liwenjun_1
2026-06-27 17:47:42 [pid=976339] [INFO] (liwenjun_1, liwenjun_2, 2026-06-27 17:47:42, sent)
2026-06-27 17:47:42 [pid=976339] [INFO] request arrival: type=chat user=liwenjun_1
2026-06-27 17:47:42 [pid=976339] [INFO] (liwenjun_1, liwenjun_2, 2026-06-27 17:47:42, sent)
2026-06-27 17:47:43 [pid=976339] [INFO] request arrival: type=chat user=liwenjun_1
2026-06-27 17:47:43 [pid=976339] [INFO] (liwenjun_1, liwenjun_2, 2026-06-27 17:47:43, sent)
2026-06-27 17:47:43 [pid=976339] [INFO] request arrival: type=chat user=liwenjun_1
2026-06-27 17:47:43 [pid=976339] [INFO] (liwenjun_1, liwenjun_2, 2026-06-27 17:47:43, sent)
2026-06-27 17:47:45 [pid=976339] [INFO] request arrival: type=chat user=liwenjun_1
2026-06-27 17:47:45 [pid=976339] [INFO] (liwenjun_1, liwenjun_2, 2026-06-27 17:47:45, sent)
2026-06-27 17:47:46 [pid=976339] [INFO] request arrival: type=logout user=liwenjun_2
2026-06-27 17:47:46 [pid=976339] [INFO] (liwenjun_2, logout, 2026-06-27 17:47:46)
2026-06-27 17:47:47 [pid=976339] [INFO] request arrival: type=chat user=liwenjun_1
2026-06-27 17:47:47 [pid=976339] [INFO] (liwenjun_1, liwenjun_2, 2026-06-27 17:47:47, pending)
2026-06-27 17:47:48 [pid=976339] [INFO] request arrival: type=login user=liwenjun_2
2026-06-27 17:47:48 [pid=976339] [INFO] (liwenjun_2, login, 2026-06-27 17:47:48) online=3
2026-06-27 17:47:48 [pid=976339] [INFO] (liwenjun_1, liwenjun_2, 2026-06-27 17:47:47, sent) [offline push]
2026-06-27 17:47:48 [pid=976339] [INFO] pushed 1 offline message(s) to liwenjun_2
[liwenjun2023150001@euler-container-30001 ChatRoom]$
```

图 3.5a server.log 记录注册、登录、发信、退出与离线回推

4. 聊天机器人管理器设计、实现与验证（20 分）

统一说明：<u>1）本题各小题中要求截图代码的，都需要结合截图讨论数据结构设计、代码线程安全与运行性能；2）如下所有机器人用户操作，如上面客户端用户一样，服务器都将作相应的日志记录</u>。

4.1. **（7 分）增加 x 个机器人，截图展示相应代码：**每个新增机器人和客户端用户一样先注册、登录系统（用户名、密码随机生成），名为“学生姓名拼音_1”的用户可以看到所有在线机器人用户并随机向其中一个聊天机器人发信息，收到信息的聊天机器人按如下规则返回信息：“幸会，学生姓名拼音_1，很高兴认识您”。

增加机器人：/bot add <x> 让服务器新增 x 个聊天机器人，每个机器人像普通客户端一样“注册+登录”（用户名、密码随机生成）；姓名_1 能在在线名单看到机器人并向其发消息，机器人按固定话术回复。

```c
/* 客户端:/bot add|del <x> 复用发消息通道,发往特殊目标 __botmgr__ */
static void handle_bot_command(const char *args)
{
    char op[8] = {0};  int x = -1;
    if (sscanf(args, "%7s %d", op, &x) != 2 ||
        (strcmp(op, "add") && strcmp(op, "del")) || x < 0) {
        printf("Usage: /bot add <x> | /bot del <x>\n");
        return;
    }
    char text[CHAT_TEXT_LEN];
    snprintf(text, sizeof(text), "%s %d", op, x);
    send_chat_request(CHAT_BOTMGR_TARGET, text);   /* 不另开第 5 个 FIFO */
}
```

图 4.1a 客户端 /bot 命令：复用发消息通道发往特殊目标 __botmgr__，不另开第 5 个 FIFO

```c
/* 机器人管理是"在线客户端用户"的动作:发起者须在线、非机器人、有私有 FIFO */
if (!user_store_lookup_online_client_fifo(&g_store, req->from, from_fifo, n)) {
    reply_to_fifo(fallback, 0, "bot manager requires login");
    return;
}
if (strcmp(op, "add") == 0) {
    for (int i = 0; i < x; i++) {
        char name[CHAT_NAME_LEN];
        if (user_store_add_bot(&g_store, name, sizeof(name), now) != USER_STORE_OK)
            break;
        log_message("INFO", "(%s, register, %s) [bot]", name, tbuf);
        log_message("INFO", "(%s, login, %s) [bot]", name, tbuf);  /* 注册即登录 */
    }
    broadcast_online_list();          /* 所有人刷新名单,看到新机器人上线 */
    reply_to_fifo(from_fifo, 1, ack); /* "added N bot(s): ..." */
}
```

图 4.1b do_bot_manager() 增加分支：校验发起者在线身份后，循环创建机器人并广播名单

```c
UserStoreStatus user_store_add_bot(UserStoreHandle *h, char *out_name,
                                   size_t out_sz, long now)
{
    su_lock(h);
    do {                                /* 随机生成不重复的机器人用户名 */
        unsigned long seq = ++s->bot_seq;
        snprintf(name, sizeof(name), "lwjbot_%lu_%u", seq, lcg_next(s) % 1000u);
    } while (find_locked(s, name) != -1 && ++tries < 64);
    snprintf(pw, sizeof(pw), "bp%u%u", lcg_next(s), lcg_next(s));  /* 随机密码 */
    ChatUserRecord *r = &s->users[s->user_count++];
    copy_string(r->username, sizeof(r->username), name);
    r->fifo[0] = '\0';                  /* 机器人没有客户端 FIFO */
    r->online = 1;  r->is_bot = 1;  r->login_time = now;
    su_unlock(h);
}
```

图 4.1c　user_store_add_bot()：随机生成不重复用户名与密码，机器人 online=1、is_bot=1、无 FIFO

```c
static void do_chat_to_bot(const char *from, const char *from_fifo,
                           const char *botname, const ChatSendInfo *info)
{
    if (!info->target_online) {        /* 机器人离线:直接丢弃,不进离线队列 */
        reply_to_fifo(from_fifo, 0, "bot is offline; message discarded (no offline queue for bots)");
        return;
    }
    user_store_increment_send(&g_store, from, botname);
    ChatPacket p;
    char reply[CHAT_TEXT_LEN];
    snprintf(reply, sizeof(reply), "幸会，%s，很高兴认识您", from);  /* 固定话术 */
    make_packet(&p, CHAT_PACKET_MESSAGE, 1, botname, reply);
    send_packet(from_fifo, &p);        /* 代机器人回信给发送者 */
}
```

图 4.1d　do_chat_to_bot()：在线机器人由服务器代发固定话术“幸会，<用户>，很高兴认识您”

复用通道与越权防护：机器人管理**复用发消息 FIFO**，以保留串 __botmgr__ 作为目标来区分，避免为管理再开一个公共 FIFO；发起者必须是“在线 + 非机器人 + 有私有 FIFO”的真实用户（user_store_lookup_online_client_fifo），防止越权；机器人无客户端进程，故 fifo 为空，其回复由服务器**代发**。

输出说明：姓名_1 执行 /bot add 2 收到 added 2 bot(s): lwjbot_…，在线名单随即出现这两个机器人；姓名_1 向某机器人发消息后，立即收到 [lwjbot_…] 幸会，姓名_1，很高兴认识您。

bot-add-list

```
[liwenjun2023150001@euler-container-30001 ChatRoom]$ ./bin/chatclient config/chatserver.conf liwenjun_1 pw1
client fifo: /home/liwenjun2023150001/Client/fifo/liwenjun_1
commands:
  /register             register current username and password
  /login                login current username and password
  /logout               logout current session (process keeps running)
  /send <user> <text>   send text to another online user (or bot)
  /bot add <x>          ask server to add x chat robots
  /bot del <x>          ask server to remove x online chat robots
  /quit                 exit client
chat> /login
chat>
[server] OK: login ok; online 1: liwenjun_1
chat>
[online 1] liwenjun_1
chat>
[system] liwenjun_2 is now online

[online 2] liwenjun_1,liwenjun_2
chat>
[system] liwenjun_3 is now online
chat>
[online 3] liwenjun_1,liwenjun_2,liwenjun_3
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2*
chat>
[system] liwenjun_2 has logged out

[online 2] liwenjun_1,liwenjun_3
chat> /send liwenjun_2 Hi, let's play badminton?
chat>
[server] ERROR: target offline; message stored as pending
chat>
[system] liwenjun_2 is now online

[online 3] liwenjun_1,liwenjun_2*,liwenjun_3
chat> /bot add 2
chat>
[online 5] liwenjun_1,liwenjun_2*,liwenjun_3,lwjbot_1_631,lwjbot_2_579
chat>
[server] OK: added 2 bot(s): lwjbot_1_631,lwjbot_2_579
chat>
```

图 4.1e-1 增加 2 个机器人后在线名单出现机器人用户

```
bot-fixed-reply
```

```
  /register             register current username and password
  /login                login current username and password
  /logout               logout current session (process keeps running)
  /send <user> <text>   send text to another online user (or bot)
  /bot add <x>          ask server to add x chat robots
  /bot del <x>          ask server to remove x online chat robots
  /quit                 exit client
chat> /login
chat>
[server] OK: login ok; online 1: liwenjun_1
chat>
[online 1] liwenjun_1
chat>
[system] liwenjun_2 is now online

[online 2] liwenjun_1,liwenjun_2
chat>
[system] liwenjun_3 is now online
chat>
[online 3] liwenjun_1,liwenjun_2,liwenjun_3
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2*
chat>
[system] liwenjun_2 has logged out

[online 2] liwenjun_1,liwenjun_3
chat> /send liwenjun_2 Hi, let's play badminton?
chat>
[server] ERROR: target offline; message stored as pending
chat>
[system] liwenjun_2 is now online

[online 3] liwenjun_1,liwenjun_2*,liwenjun_3
chat> /bot add 2
chat>
[online 5] liwenjun_1,liwenjun_2*,liwenjun_3,lwjbot_1_631,lwjbot_2_579
chat>
[server] OK: added 2 bot(s): lwjbot_1_631,lwjbot_2_579
chat> /send lwjbot_1_631 hello robot
chat>
[lwjbot_1_631] 幸会，liwenjun_1，很高兴认识您
chat>
[server] OK: bot replied
chat>
```

图 4.1e-2 向机器人 lwjbot_1_631 发消息并收到固定回复

4.2. **（7 分）减少 x 个机器人，截图展示相应代码：**随机选择 x 个机器人用户，向服务器发送退出请求，名为“学生姓名拼音_1”的用户可以看到相应机器人用户退出的信息。特别提醒：有了机器人用户，离线用户信息的管理需要进一步考虑合理性，因为机器人用户的用户

名是随机生成的，下次增加就不再是原来哪些机器人了。

减少机器人：`/bot del <x>` 随机选 x 个在线机器人向服务器发退出请求，姓名_1 能看到相应机器人退出的信息。

```
else {   /* del:随机选 x 个在线机器人下线 */
    char names[CHAT_MAX_USERS][CHAT_NAME_LEN];
    int n = user_store_pick_online_bots(&g_store, x, names, CHAT_MAX_USERS, now);
    for (int i = 0; i < n; i++) {
        log_message("INFO", "(%s, logout, %s) [bot]", names[i], tbuf);
        char sys[128];
        snprintf(sys, sizeof(sys), "robot %s has logged out", names[i]);
        broadcast_system(sys, NULL);      /* 通知在线用户该机器人退出 */
    }
    broadcast_online_list();
    reply_to_fifo(from_fifo, 1, ack);     /* "removed N bot(s): ..." */
}
```

图 4.2a　do_bot_manager() 减少分支：随机选中的机器人逐个置离线并广播退出

```
int user_store_pick_online_bots(UserStoreHandle *h, int x,
                                char names[][CHAT_NAME_LEN], int max, long now)
{
    su_lock(h);
    int m = 0;
    for (int i = 0; i < s->user_count; i++)            /* 收集在线机器人下标 */
        if (s->users[i].online && s->users[i].is_bot) idxs[m++] = i;
    int want = (x < m) ? x : m;
    for (int k = 0; k < want; k++) {                   /* 部分洗牌:等概率抽 want 个 */
        int r = k + (int)(lcg_next(s) % (unsigned)(m - k));
        int bi = idxs[r];  idxs[r] = idxs[k];  idxs[k] = bi;
        s->users[bi].online = 0;  s->users[bi].logout_time = now;
        copy_string(names[n++], CHAT_NAME_LEN, s->users[bi].username);
    }
    su_unlock(h);
    return n;
}
```

图 4.2b　user_store_pick_online_bots()：部分 Fisher–Yates 洗牌，O(want) 等概率随机抽取并下线

离线管理的合理性：机器人用户名是随机生成的，下次增加就不再是原来那些机器人。因此**机器人不保留离线消息队列**：`do_chat_to_bot` 对离线机器人直接丢弃消息（图 4.1d），不像真人那样暂存——否则会给“再也不会回来的随机名”堆积永远无法送达的离线消息。随机抽取用**部分洗牌**实现，只洗前 `want` 个、复杂度 `O(want)` 且每个在线机器人被选中的概率相等。

输出说明：姓名_1 执行 `/bot del 1` 收到 `removed 1 bot(s): lwjbot_…`；所有在线用户收到 `[system] robot lwjbot_… has logged out` 并看到刷新后的在线名单。

bot-del-operator

```
  /bot del <x>          ask server to remove x online chat robots
  /quit                 exit client
chat> /login
chat>
[server] OK: login ok; online 1: liwenjun_1
chat>
[online 1] liwenjun_1
chat>
[system] liwenjun_2 is now online

[online 2] liwenjun_1,liwenjun_2
chat>
[system] liwenjun_3 is now online
chat>
[online 3] liwenjun_1,liwenjun_2,liwenjun_3
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2
chat> /send liwenjun_2 How are you, liwenjun_2
chat>
[server] OK: message sent to liwenjun_2*
chat>
[system] liwenjun_2 has logged out

[online 2] liwenjun_1,liwenjun_3
chat> /send liwenjun_2 Hi, let's play badminton?
chat>
[server] ERROR: target offline; message stored as pending
chat>
[system] liwenjun_2 is now online

[online 3] liwenjun_1,liwenjun_2*,liwenjun_3
chat> /bot add 2
chat>
[online 5] liwenjun_1,liwenjun_2*,liwenjun_3,lwjbot_1_631,lwjbot_2_579
chat>
[server] OK: added 2 bot(s): lwjbot_1_631,lwjbot_2_579
chat> /send lwjbot_1_631 hello robot
chat>
[lwjbot_1_631] 幸会，liwenjun_1，很高兴认识您
chat>
[server] OK: bot replied
chat> /bot del 1
chat>
[system] robot lwjbot_1_631 has logged out

[online 4] liwenjun_1,liwenjun_2*,liwenjun_3,lwjbot_2_579

[server] OK: removed 1 bot(s): lwjbot_1_631
chat>
```

图 4.2c-1 发起删除 1 个机器人后在线名单更新

```
bot-del-broadcast-liwenjun-2

[liwenjun2023150001@euler-container-30001 ChatRoom]$ ./bin/chatclient config/chatserver.conf liwenjun_2 pw2
client fifo: /home/liwenjun2023150001/Client/fifo/liwenjun_2
commands:
  /register             register current username and password
  /login                login current username and password
  /logout               logout current session (process keeps running)
  /send <user> <text>   send text to another online user (or bot)
  /bot add <x>          ask server to add x chat robots
  /bot del <x>          ask server to remove x online chat robots
  /quit                 exit client
chat> /login
chat>
[server] OK: login ok; online 2: liwenjun_1,liwenjun_2
chat>
[online 2] liwenjun_1,liwenjun_2
chat>
[system] liwenjun_3 is now online

[online 3] liwenjun_1,liwenjun_2,liwenjun_3
chat>
[liwenjun_1] How are you, liwenjun_2
chat>
[liwenjun_1] How are you, liwenjun_2
chat>
[liwenjun_1] How are you, liwenjun_2
chat>
[liwenjun_1] How are you, liwenjun_2
chat>
[liwenjun_1] How are you, liwenjun_2
chat>
[liwenjun_1] How are you, liwenjun_2
chat>
[liwenjun_1] How are you, liwenjun_2
chat> /logout
chat>
[server] OK: logout ok
chat> /login
chat>
[server] OK: login ok; online 3: liwenjun_1,liwenjun_2,liwenjun_3
chat>
[offline from liwenjun_1 @ 2026-06-27 17:47:47] Hi, let's play badminton?

[online 3] liwenjun_1,liwenjun_2,liwenjun_3
chat>
[online 5] liwenjun_1,liwenjun_2,liwenjun_3,lwjbot_1_631,lwjbot_2_579
chat>
[system] robot lwjbot_1_631 has logged out

[online 4] liwenjun_1,liwenjun_2,liwenjun_3,lwjbot_2_579
chat>
```

图 4.2c-2 终端 2 收到机器人退出系统消息

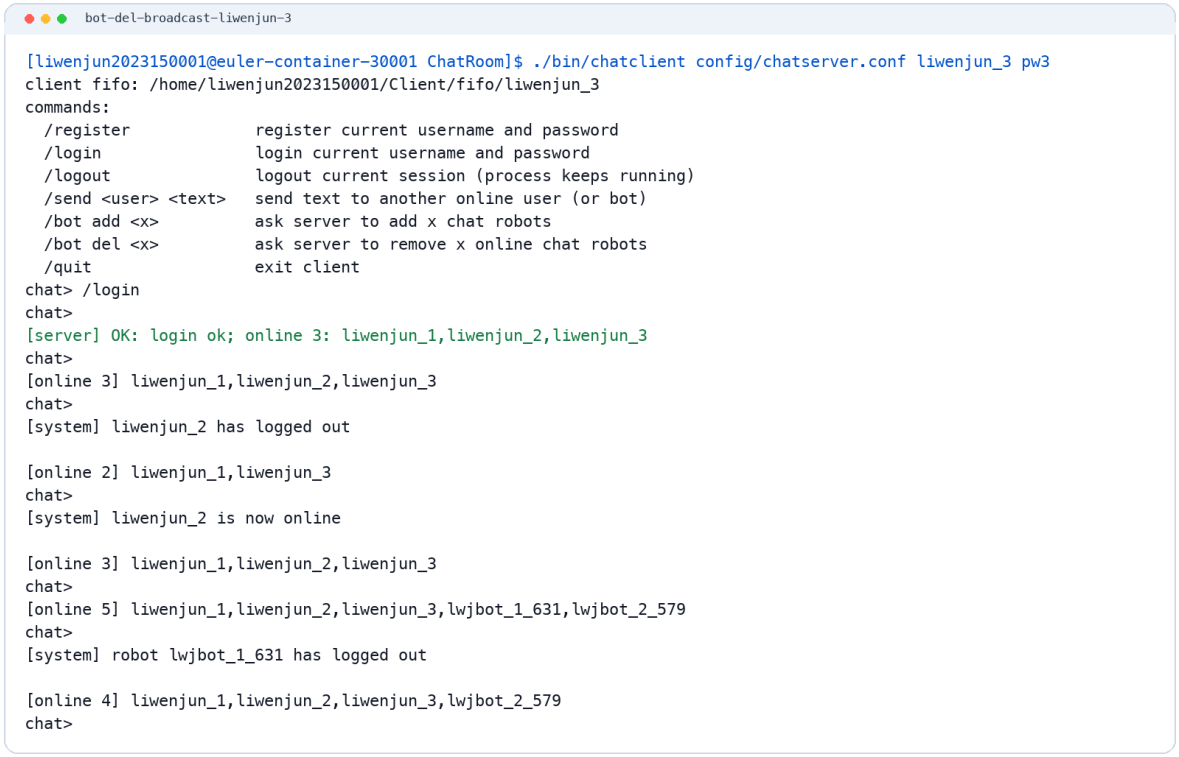

图 4.2c-3 终端 3 收到机器人退出系统消息

4.3. （6 分）请运行以下 Shell 命令并截取运行结果：

$cat ~/log/chat-logs/server/server.log

截图展示相应机器人注册、登录、发信息和退出等日志信息。

机器人日志：机器人的每一步操作都与真人一样写入 server.log，并用 [bot] 标注以便区分。cat server.log 可看到形如 (lwjbot_3_217, register, ...) [bot]、(..., login, ...) [bot]、(姓名_1, lwjbot_3_217, ..., sent) [to bot]、(lwjbot_3_217, 姓名_1, ..., sent) [bot reply]、(lwjbot_..., logout, ...) [bot] 等记录，完整覆盖机器人注册、登录、收发消息与退出全过程。

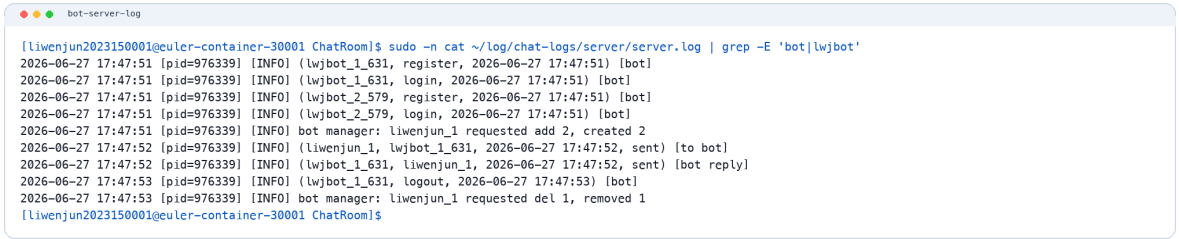

图 4.3a server.log 中机器人注册、登录、发信与退出记录